

App para trackear macros y pesos.

Stack

Backend

- Java 26 + Spring Boot 4.0.6
- Spring Data JPA + Hibernate 7.12
- Spring Security + JWT
- Spring Web
- Spring Validation
- PostgreSQL18

Frontend

- Astro 6.3
- React 18.3.1
- Tailwind CSS 3.4.17
- Tremor 3.18.7 para gráficas del dashboard

Requisitos funcionales

RF01 - Gestión de usuarios

- Registro con email + contraseña
- Login con JWT (access token 30 min + refresh token 7 días)
- Logout (invalidar refresh token en servidor)
- Cada usuario ve **exclusivamente** sus propios datos

- Chatbot que pueda leer y escribir esa bbdd
- Login con gestión de usuarios.

RF02 - Registro diario

- Formulario con los siguientes campos:
 - **Fecha** (date picker, por defecto ayer)
 - **Peso**
 - %Graso
 - Kcal gastadas
 - Kcal ingeridas
 - Seleccionable de tipo de deporte/descanso (Ver como añadir tipos de deporte (gym, carrera suave zona2, series, intervalos...) que cada usuario pueda añadir los tipos de entreno que quiera)
 - Tiempo total de ejercicio (min)
 - Notas
- Selector rápido de fecha (hoy / ayer / calendario)
- Validación en tiempo real (campos en rojo si son inválidos) Ej: introduce letras en el campo de los min de ejercicio.
- Confirmación al guardar o al detectar datos existentes, en caso de que ya existan, dar opción de sobrescribir.

Chatbot

- Interfaz de chat en la derecha (fijo) con historial persistente por usuario.
- Indicador de "Pensando..." mientras la IA procesa
- Soporte para streaming de respuestas (el texto aparece progresivamente)
- Indicador de modelo activo y proveedor (Deepseek-V4-pro · Deepseek)

- Boton para eliminar el historial del chat (Con confirmación)

Chatbot: arquitectura detallada

Flujo de una consulta

1. Usuario escribe mensaje → `POST /api/v1/chat/messages`
2. Backend comprueba autenticación (JWT)
3. Backend construye el system prompt con:
 - a. Rol: "Eres un entrenador y nutricionista profesional..." (Planear system prompt).
 - b. Tools disponibles: `get_records`, `create_record`, `update_record`, `get_stats` ...
4. Backend envía mensaje + tools al proveedor IA usando la api key almacenada del usuario.
5. Si la IA solicita ejecutar una tool → backend ejecuta la consulta SQL (limitada al `user_id` del token) → devuelve resultado a la IA
6. La IA genera respuesta final → backend transmite en streaming (SSE) al frontend
7. Se almacena el historial de mensajes en BBDD para mantener contexto entre sesiones

Pestañas de la app

1. Home: dashboard con la info relevante (gráficas con el histórico de pesos, %graso y medidas corporales (para este apartado añadir maniquí en posición de estrella mostrando las medidas))...
2. Introducción de datos diaria: formulario con todos los datos que se almacenan en bbdd, debe comprobar que ya existan datos en esa fecha, en ese caso pedir confirmación para pisar datos (sobrecribir) o cancelar.
3. Configuración: donde se ajustaran datos del chatbot y usuario (cerrar sesiones, etc)

Chatbot

1. Api key openai compatible.
2. Selector de modelos segun proveedor. Si mete la api key de openai que salgan ciertos modelos de openai (los principales, excluir codex, mini o versiones inutiles para esto), si mete la de openrouter que salgan todos los que ofrezca.

Futuribles

- Integración con Apple Health, Renpho health, google health, etc para automatizar la inserción de ciertos datos (Investigar APIs y viabilidad de esto).
- Perfiles de entrenador/cliente (haría la app vendible a entrenadores).
- Subida de fotos de progreso (comparador de antes/despues en dashboard principal).